

01 — Can gas exchange explain ocean carbon storage?

Learning goals: map boxes and arrows to a conserved carbon inventory; infer background TA and distinguish a fit from a prediction; distinguish equilibrium controls from rate controls.

Suppose you are testing your first ESBMTK atmosphere–ocean model. You build two reservoirs: a finite atmosphere and one well-mixed ocean box.

Picture the initial ocean as water containing only dissolved NaCl, with **TA = 0**. Dissolve a trace of CO₂ to give it a small positive DIC concentration; TA remains zero. Place almost all the remaining carbon in the atmosphere, then let the two reservoirs exchange CO₂. Nothing enters or leaves the system. This is a fictional redistribution experiment, not a history of ocean formation.

We use two **reference values** throughout this notebook: atmospheric dry-air xCO₂ = **280 ppm** and ocean DIC = **2040 μmol/kg**.

Choose the **total carbon inventory** to equal the combined atmosphere–ocean inventory at these reference values. Your hypothesis is: **if the model is coded correctly, gas exchange should recover both reference values**. You will predict, run, and assess that claim.

Provisional time: 40 minutes. Diagram and prediction (10); diagnosis, TA inference, linked chemistry views and pCO₂–DIC curves (20); model comparisons and explanation (10). Reuse your PyCO₂SYs skills from 00 to infer TA, then calculate seawater pCO₂ across a supplied DIC range. A guided 3–5-minute surface/contour/slice introduction replaces part of the sensitivity explanation within that provisional 20-minute block; pilot it before treating the allocation as established. Model construction, loops, conversions, plotting and budget checks are supplied. Give short answers.

Optional coding reference: [From conceptual model to code \(two-page handout\)](#). Use it for a reminder; the syntax needed here is introduced below. Code labels distinguish **Choose and explain** (your scientific choices), **Understand and run** (supplied model steps), and **Supplied implementation** (support code). This practical is ungraded. Syntax memorisation is not required: refer to the examples and focus on connecting the scientific assumptions to the code. See [teaching goals and timing](#).

Reading key: **Key term** = concept to notice; **Question** = student prompt. Instructor answers use labelled purple panels in the instructor sheet. These reading cues complement the code labels above.

LECTURER REVIEW | INSTRUCTOR EDITION

Executed in ESBMTK314. All instructor solutions, code, execution outputs and optional disclosures are included. No Jupyter installation is needed to read this PDF.

Source: 01_single_box_air_sea_CO2.ipynb

Execution: 2026-09-24. The source notebooks are unchanged.

```
# Supplied implementation: run this support code as provided.
from pathlib import Path
import sys
import numpy as np
import matplotlib.pyplot as plt
import PyCO2SYS as pyco2

ROOT = next(p for p in (Path.cwd(), *Path.cwd().parents)
             if (p / 'teaching_config.py').is_file())
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))
from teaching_config import TEACHING as config
from simple_models import (
```

```

new_model, box_parameters, box_mass_kg, connect_atmosphere,
single_box, inventories, audit, atmospheric_pco2_curve,
)
from esbmtk import initialize_reservoirs, add_carbonate_system_1
from model import run_model
from teaching_plots import (
    plot_ta_free, plot_partition_comparison, plot_equilibrium_curves,
)

```

1. Specify the system

```

Atmosphere (xCO2; GasReservoir)
    ↓ J_gas,in      ↑ J_gas,out
    gas exchange: carbon only
Ocean (DIC, TA; initialize_reservoirs)
    carbonate system 1 → aqueous CO2

```

Both reservoirs are inside the **system boundary**. Write each carbon budget as **inputs minus outputs**. The subscripts *atm* and *ocn* denote atmosphere and ocean. Invasion, $J_{gas,in}(t)$, enters the ocean; outgassing, $J_{gas,out}(t)$, leaves it:

$$\frac{dC_{atm}(t)}{dt} = J_{gas,out}(t) - J_{gas,in}(t),$$

$$m_{ocn} \frac{dDIC_{ocn}(t)}{dt} = J_{gas,in}(t) - J_{gas,out}(t).$$

Each flux is in mol C/yr. Ocean DIC is in mol/kg and water mass m_{ocn} is in kg. A transfer leaves one reservoir and enters the other by the same amount. Gas exchange carries no TA, so the ocean TA inventory has zero tendency.

Quantity	Role in this experiment
280 ppm dry-air xCO ₂ ; 2040 μmol/kg DIC	Observed comparison targets
Ocean area and depth; atmospheric size; T/S/P	Independent inputs
Combined carbon inventory calculated from the targets	Target-derived constraint
Almost all carbon initially in the atmosphere	Fictional initial partition
Initial TA = 0	Physical assumption to test

The ocean has area 3.60×10^{14} m² and depth 3750 m. Uniform **T = 16 °C, S = 35 and P = 0 bar** and carbonate choices come from `teaching_config.py`. The NaCl picture motivates the initial DIC and TA; the calculation uses these supplied seawater chemistry settings. ESBMTK uses bar and PyCO2SYS uses dbar; the configuration handles the conversion. Density ρ comes from ESBMTK at the same conditions.

With ocean volume V_{ocn} and atmospheric size N_{atm} , the total carbon is

$$C_0 = N_{atm}(280 \times 10^{-6}) + \rho V_{ocn}(2040 \times 10^{-6}).$$

The two factors convert ppm to mol/mol and μmol/kg to mol/kg, respectively. No geometry is fitted to a carbon ratio.

Reading the settings. `config` is the shared configuration imported above. The dot retrieves a named setting:

`config.ocean_volume_m3` is the ocean volume in m³. Names ending in `_umol_kg` or `_ppm` indicate the units. Some settings, such as `config.pyco2`, collect several named values in a Python **dictionary**. The next cell displays these settings; you do not need to edit the configuration file.

```

# Understand and run: trace this supplied step and interpret its evidence.
print('Shared chemistry:', config.pyco2)
print('Ocean volume (m3):', config.ocean_volume_m3)
print('ESBMTK density (kg/m3):', config.density_kg_m3)
print('Target-derived total carbon (mol):', config.total_carbon_mol)

```

```
INITIAL_DIC = 0.01 # umol/kg, small positive initial concentration
FIRST_TA = 0.0 # umol/kg: no background alkalinity
```

Shared chemistry: {'opt_k_carbonic': 10, 'opt_pH_scale': 2, 'opt_buffers_mode': 1, 'temperature': 16.0, 'salinity': 35.0, 'pressure': 0.0}

Ocean volume (m3): 1.35e+18

ESBMTK density (kg/m3): 1025.7484950008022

Target-derived total carbon (mol): 2.8744713552322094e+18

2. Build the supplied model

2.1 A finite atmosphere and an initial carbon partition

The atmosphere contains a fixed $N_{atm} = 1.77 \times 10^{20}$ mol of air. Its **dry-air CO2 mole fraction**, $x_{CO_2}(t)$, changes during exchange; $10^6 x_{CO_2}(t)$ gives ppm. Atmospheric carbon is $C_{atm}(t) = N_{atm} x_{CO_2}(t)$ mol C. No atmospheric height or volume is needed.

We initialize ocean DIC at **0.01 $\mu\text{mol/kg}$** , a **small positive initial concentration**. This lets the carbonate calculation start away from exactly zero DIC; it does not supply alkalinity. The remaining carbon starts in the atmosphere. With $m_{ocn} = \rho V_{ocn}$,

$$C_{atm}(0) = C_0 - m_{ocn} DIC_{ocn}(0), \quad x_{CO_2}(0) = \frac{C_0 - m_{ocn} DIC_{ocn}(0)}{N_{atm}}.$$

Use DIC in mol/kg in these equations. The resulting initial atmosphere is approximately **16,240 ppm**. The **280 ppm reference defines part of**; it is neither the initial concentration nor a fixed atmospheric boundary value.

2.2 Two directional fluxes, one net exchange

The **gas transfer velocity (piston velocity)**, v , describes how readily CO2 crosses the air–sea interface. It has units of length/time; it is not the vertical speed of a water parcel. Here we prescribe $v = 4 \text{ m/day}$ over ocean area A .

Multiplying v by density and the aqueous-CO2 concentration difference gives the net transfer per unit area. The two directional fluxes are

$$J_{gas,in}(t) = A v \rho \beta pCO_{2,atm}(t), \quad J_{gas,out}(t) = A v \rho [CO_2]_{aq}(t).$$

The solubility coefficient β converts atmospheric pCO2 into a dissolved CO2 concentration. Both $\beta pCO_{2,atm}$ and $[CO_2]_{aq}$ are in mol/kg. With A in m², v converted to m/yr and ρ in kg/m³, both fluxes are in mol C/yr.

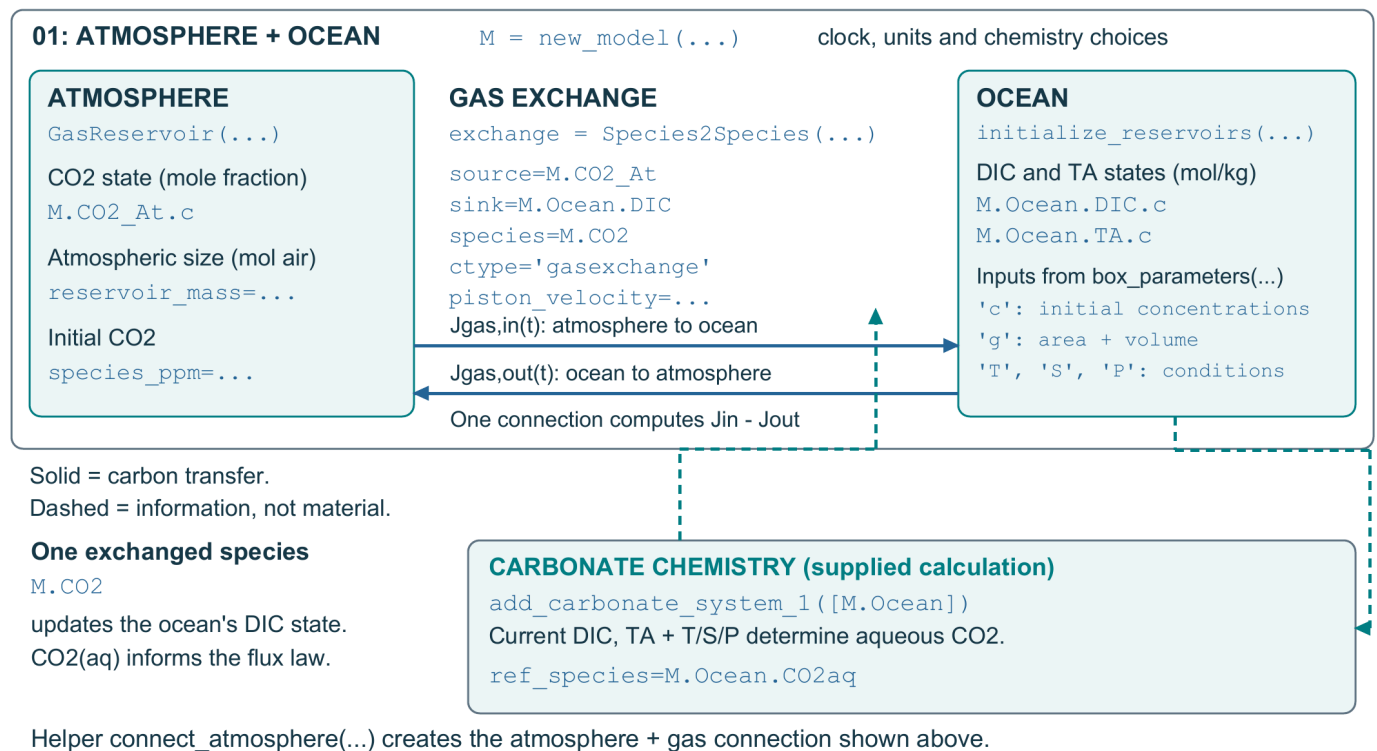
Invasion depends on atmospheric CO2 and the prescribed exchange and solubility settings, independently of ocean DIC and TA. **Outgassing** depends on aqueous CO2, which carbonate chemistry calculates from ocean DIC and TA. The ocean gains their difference; the atmosphere loses the same amount:

$$J_{gas}(t) = J_{gas,in}(t) - J_{gas,out}(t).$$

Positive J_{gas} means net ocean uptake; negative J_{gas} means net outgassing. **At equilibrium** the two directional rates balance, although both remain nonzero. The closed budget is $C_{atm}(t) + m_{ocn} DIC_{ocn}(t) = C_0$.

Units reference. This follows the solubility-times-pCO2 notation in [the ESBMTK paper, equation 6](#). Our atmospheric state is dry-air mole fraction, not partial pressure. The supplied [connect_atmosphere](#) function handles gas-convention and unit conversions through `solubility` and `scale`, using PyCO2SYS and ESBMTK density. You do not need to implement these conversions. The 0 bar setting is seawater pressure, not atmospheric pressure; solubility uses the shared 16 °C and salinity 35.

2.3 Map the diagram to objects



Helper `connect_atmosphere(...)` creates the atmosphere + gas connection shown above.

Read the code labels on the diagram alongside the construction below:

- `Model` supplies the clock and units. `M.co2` is a species definition.
- `initialize_reservoirs` creates the ocean DIC and TA states. `box_parameters` supplies geometry (*g*), conditions (*T*, *S*, *P*) and initial concentrations (*c*).
- `add_carbonate_system_1` calculates `M.Ocean.CO2aq` from DIC and TA. **This diagnostic** informs gas exchange; it is not another carbon inventory.

Solid arrows transfer carbon. Dashed arrows supply information to a calculation. The `.c` attribute stores concentration values: mol/kg for ocean states and mol/mol for atmospheric CO2. Indices `[0]` and `[-1]` select the first and last saved values.

The supplied setup registers species definitions before any ocean reservoirs are created. You only prescribe the ocean DIC and TA states here.

Chemistry reference (optional). `new_model` registers Carbon, Boron, Hydrogen and miscellaneous species definitions. Seawater initialization supplies background boron through `PyCO2SYS`; carbonate-system code initializes and updates H^+ and aqueous CO2 from DIC/TA. These are not extra transported reservoirs to construct. The miscellaneous sediment-variable definitions are unused in 01; they activate no sediment processes.

In `{'Ocean': box_parameters(...)}`, the quoted name is a dictionary **key**; the function result after the colon is its **value**. `initialize_reservoirs` uses that pair to create the named box from its supplied properties.

First create the ocean and its carbonate chemistry:

```
# Understand and run: trace this supplied step and interpret its evidence.
M = new_model(stop='2 kyr', max_timestep='1 yr')
initialize_reservoirs(M, {'Ocean': box_parameters(
    M, config.ocean_volume_m3, INITIAL_DIC, FIRST_TA)})
add_carbonate_system_1([M.Ocean])
```

If you use ESBMTK for your research, please cite:

Wortmann et al. 2025, <https://doi.org/10.5194/gmd-18-1155-2025>

2.4 Connect atmospheric CO2 to ocean DIC

The native constructor `Species2Species` connects two species states. `source` and `sink` set the positive direction; `ctype` selects the flux law. The `id` is a label used to identify a connection in summaries or later lookups; here `air_sea` labels this one gas connection. `connect_atmosphere` is a supplied function defined in `simple_models.py`. It creates the finite atmosphere and its gas-exchange connection. The excerpt below comes from that function: its local names `model` and `surface` correspond to `M` and `M.Ocean` in this notebook. The function supplies the parameter values and unit conversions.

```
model.air_sea_exchange = Species2Species(
    source=model.CO2_At,
    sink=surface.DIC,
    species=model.CO2,
    ctype="gasexchange",
    piston_velocity=piston_velocity,
    solubility=f"{beta_native} mol/(m**3 * atm)",
    scale=surface.swc.density / 1000.0,
    ref_species=surface.CO2aq,
    id="air_sea",
)
```

`species` identifies the exchanged gas, while `sink` identifies the ocean DIC state it changes. `ref_species` provides aqueous CO2 for the outgassing term. The `gasexchange` law calculates the signed difference of invasion and outgassing: **two conceptual arrows are represented by one connection**.

The next cell calls `connect_atmosphere` and returns the connection as `exchange`; the atmospheric state is `M.CO2_At`.

Supplied verification checks. `assert` checks whether a condition holds; `assert_allclose` checks numerical agreement within a tolerance. A failed check stops the cell with an error. These checks test the calculated result; they do not force the model to match it. Here they check the connection endpoints, ocean mass and initial TA. Later checks test conservation and calibrated agreement.

```
# Understand and run: connect the atmosphere and check the construction.
exchange = connect_atmosphere(M, [M.Ocean])
# Supplied verification: connection endpoints, ocean mass and initial TA.
assert exchange.source is M.CO2_At and exchange.sink is M.Ocean.DIC
np.testing.assert_allclose(box_mass_kg(M.Ocean),
                           config.ocean_volume_m3 * config.density_kg_m3, rtol=1e-12)
assert M.Ocean.TA.c[0] == 0
print('Initial atmosphere (ppm):', M.CO2_At.c[0] * 1e6)
print('Initial ocean carbon fraction:',
      box_mass_kg(M.Ocean) * M.Ocean.DIC.c[0] / config.total_carbon_mol)
```

Initial atmosphere (ppm): 16239.872924449308

Initial ocean carbon fraction: 4.817443964889389e-06

Before running: check the mapping and predict

Question — map and check

Point to the ocean DIC/TA states and gas connection in the code. Which quantity is calculated from DIC and TA? Add the two carbon tendencies in section 1: why do the internal transfers cancel? Why must you multiply ocean concentration by water mass when checking the carbon inventory?

Instructor answer

M.Ocean.DIC and M.Ocean.TA are states; carbonate chemistry calculates aqueous CO₂. connect_atmosphere creates the atmosphere and its gas connection. The ocean gains exactly what the atmosphere loses, so their carbon sum is constant. mol/kg times kg is mol. TA has no source or sink in this model.

Question — predict before running

Write a short prediction before running the next cell. Does specifying the combined carbon inventory guarantee recovery of both reference values? What else might determine the final atmosphere–ocean partition?

```
# Understand and run: trace this supplied step and interpret its evidence.
run_model(M)
# Supplied verification: carbon and TA conservation throughout the run.
print(audit(M))
print('TA-free result: xCO2 (ppm), DIC (umol/kg):',
      M.CO2_At.c[-1] * 1e6, M.Ocean.DIC.c[-1] * 1e6)
plot_ta_free(M, config)
```

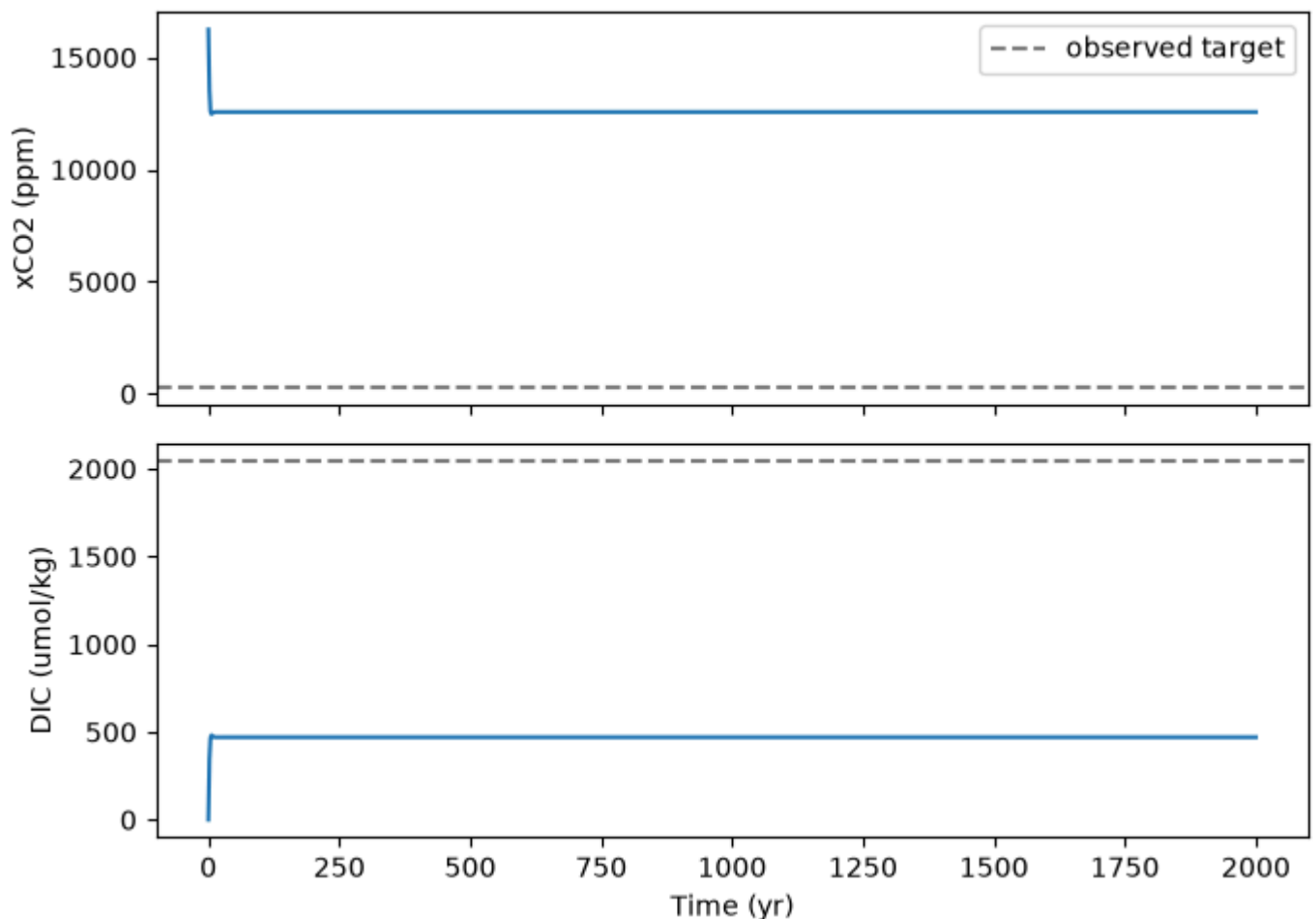
status=0

message=The solver successfully reached the end of the integration interval.

Execution took 0.73 CPU seconds, wall time = 0.73 seconds

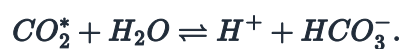
```
{'carbon_relative_error': 3.562394170796242e-16, 'ta_change_mol': 0.0}
```

TA-free result: xCO₂ (ppm), DIC (umol/kg): 12561.780484274073 470.14355510736226



After running: diagnose the result

Here CO_2^* denotes the dissolved CO_2 /carbonic-acid pool. Recall the reaction



Question — diagnose after running

Revisit your prediction using the carbon and TA budget checks. Does disagreement with the reference values demonstrate a coding error? What do passing budget checks establish, and what do they leave untested?

CO_2 enters the water and its pH changes. Why does this not create or consume TA? Explain using the paired bicarbonate and proton contributions. Which initial chemical assumption therefore persists and prevents recovery of the reference partition?

Instructor answer

Passing carbon and TA checks supports correct budget implementation. It does not prove complete code correctness or that the physical assumptions represent the ocean. The mismatch alone is therefore not evidence of a coding error.

The total carbon fixes the combined inventory, not its equilibrium partition. At fixed geometry and thermodynamic settings, that partition also depends on TA. Each bicarbonate contributes +1 equivalent to TA and the accompanying proton contributes -1: their contributions cancel. Subsequent acid–base repartitioning also conserves TA, although pH changes. Gas exchange adds DIC but no TA, so the initial TA = 0 persists. No process in this model can generate the missing alkalinity. A small initial DIC does not require zero TA; these are separate initial assumptions. Section 3 tests how changing TA changes the relation between DIC and seawater pCO₂.

3. Infer TA, explain the partition, then test the revised model

PyCO₂SYs calculates carbonate-equilibrium states; ESBMTK follows carbon transfer through time. First use the reference DIC and atmospheric xCO₂ to infer TA. Then use DIC and TA to calculate seawater pCO₂ and explain the contrasting carbon partitions.

3.1 Infer TA from the reference state

Question — calculate and explain

Exercise 01.1 — reuse your PyCO₂SYs skills from 00. Calculate the TA required for seawater with **DIC = 2040 $\mu\text{mol/kg}$** to be in equilibrium with atmospheric **dry-air xCO₂ = 280 ppm**. Choose the input types, write the PyCO₂SYs call and extract TA. Save the result as **inferred_ta, in $\mu\text{mol/kg}$** .

Use `config.target_dic_umol_kg`, `config.target_xco2_ppm` and the supplied settings `**config.pyco2`. The double star passes the dictionary's entries as named arguments to the function. These settings include **16 °C**, salinity 35, pressure 0 dbar and the shared carbonate choices. Do not carry over 00's 15 °C baseline. Consult your work from 00 and the [PyCO₂SYs input and result documentation](#) as needed. Retrieve a result using `result['key']`, where `key` is its exact name in the documentation. Dry-air xCO₂ in ppm differs from pCO₂ in μatm .

After calculating TA, explain what was fitted and why the result is **not an independent prediction** of ocean TA.

Instructor answer

TA is inferred from the two reference targets under the prescribed conditions. It is a fitted background parameter, not an independent prediction. Recovering the same targets in ESBMTK checks consistency between the implementations and the prescribed carbon inventory; it does not independently validate the model.

```
# Choose and explain: calculate TA from the two reference targets.
# Save inferred_ta in umol/kg; use the shared settings in config.pyco2.
# BEGIN SOLUTION
dic_input_type = 2
air_input_type = 9
reference = pyco2.sys(par1=config.target_dic_umol_kg, par1_type=dic_input_type,
                    par2=config.target_xco2_ppm, par2_type=air_input_type, **config.pyco2)
inferred_ta = float(reference['alkalinity'])
```



```
# END SOLUTION
print('Inferred TA (umol/kg):', inferred_ta)
```

Inferred TA (umol/kg): 2348.2362117173566

3.2 DIC and TA constrain carbonate chemistry

At fixed temperature, salinity and pressure, DIC and TA determine seawater $p\text{CO}_2$. The supplied explorer puts **DIC on x, TA on y and $p\text{CO}_2$ on the vertical axis**. Each point is a possible carbonate-equilibrium state, not a time step. Its $p\text{CO}_2$ is the atmospheric partial pressure that would balance gas exchange with that water; it need not equal the **actual** atmosphere in our closed model. Use the shared **16 °C, salinity 35, 0 bar** conditions throughout this section.

Guided exploration (3–5 minutes, supplied code).

1. **Rotate surface:** drag the surface, or focus it and use arrow keys. Start near the inferred reference state; height and colour both encode $p\text{CO}_2$.
2. **Top view / contours:** each labelled contour joins states with the same $p\text{CO}_2$. This is the $p\text{CO}_2$ version of the DIC–TA **Deffeyes diagram** on [lecture slide 16](#). That slide also shows pH contours: those require a different surface over the same DIC–TA plane.
3. **Constant-TA slice:** the vertical cutting plane intersects the surface along the blue curve. The right panel shows exactly this $p\text{CO}_2$ –DIC curve. Move the **TA slider** and follow both views. Merely looking at the whole surface from the side would not isolate one TA value.
4. Move the **DIC point** and select **Show local tangent**. The tangent is a straight line with the curve's local slope. In top view, relate this response to the $p\text{CO}_2$ change between labelled contours along a horizontal TA slice. Select **Full range, including TA = 0** to see why the much larger height range can hide near-reference curvature. Each panel's axes are labelled; do not compare apparent angles across differently scaled views.

Leave the **atm line** off until the carbon-conservation explanation below. The surface by itself supplies chemistry, not a closed-system endpoint or an equilibration time. The optional full-range view adds no extra calculation. Use the existing curve-interpretation question below to explain what you see; no separate written answer or plotting code is required here.

Run the next cell to activate the explorer in JupyterLab. It works offline with the existing course environment. **Save offline copy** downloads a standalone HTML version that can be opened in a browser. Static curves remain in the next exercise; the biological and carbonate process arrows are introduced in 03.

```
# Supplied implementation: explore linked views; chemistry and plotting are provided.
from teaching_carbonate import show_carbonate_explorer
show_carbonate_explorer(config, inferred_ta, initial_dic=INITIAL_DIC)
```

Interactive carbonate explorer: static review view. The companion [01_carbonate_explorer.html](#) opens offline in a browser without Jupyter. The PDF shows the default surface, contours and slice at the reference TA; use the companion for rotation and sliders.

DIC and TA constrain carbonate chemistry

16 °C · salinity 35 · 0 bar · carbonate constants 10 · pH scale 2

1 · Rotate surface

2 · Top view / contours

3 · Constant-TA slice

Reset

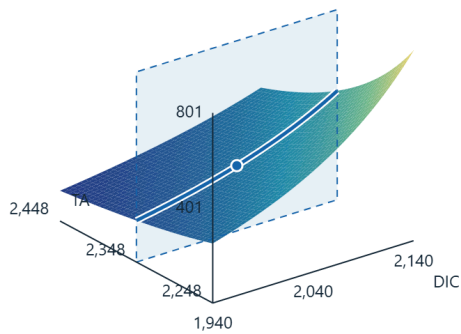
Range Near reference seawater ▼

TA 2,348.2 $\mu\text{eq/kg}$ DIC point 2,040 $\mu\text{mol C/kg}$ ☐ Show local tangent

☐ Add conserved-carbon atm line

Seawater pCO₂ surface + constant-TA plane

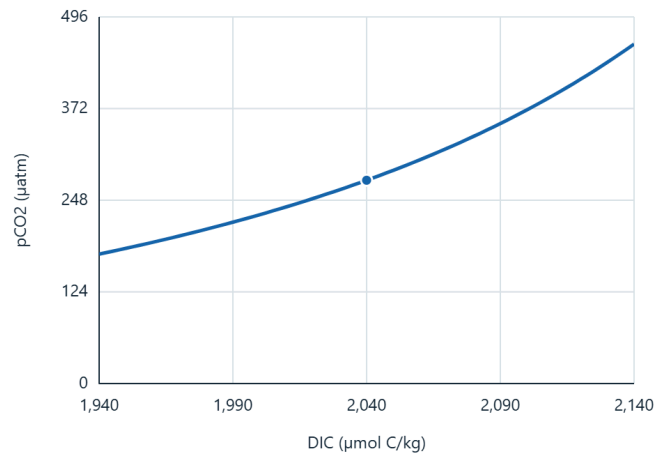
Height: pCO₂ (μatm)



DIC: $\mu\text{mol C/kg}$ · TA: $\mu\text{eq/kg}$
pCO₂ (μatm) 128 801

Drag or use arrow keys to rotate. The blue curve is the surface cut by the vertical constant-TA plane.

pCO₂-DIC at the selected TA



— ocn: selected TA

The right-hand curve has its own linear pCO₂ scale. Select Full range to include TA = 0; near-reference seawater then looks flatter.

Selected state: DIC 2,040 $\mu\text{mol C/kg}$; TA 2,348.2 $\mu\text{eq/kg}$; seawater pCO₂ 275.08 μatm . Local sensitivity at fixed TA: 1.308 $\mu\text{atm per } (\mu\text{mol C/kg})$.

Each point is a possible carbonate-equilibrium state, not a time step or necessarily air-sea equilibrium. Fixed T/S/P and chemistry choices apply to every view. Height, colour and contours encode pCO₂; pH would require a different surface.

DIC and TA constrain carbonate chemistry

16 °C · salinity 35 · 0 bar · carbonate constants 10 · pH scale 2

1 · Rotate surface

2 · Top view / contours

3 · Constant-TA slice

Reset

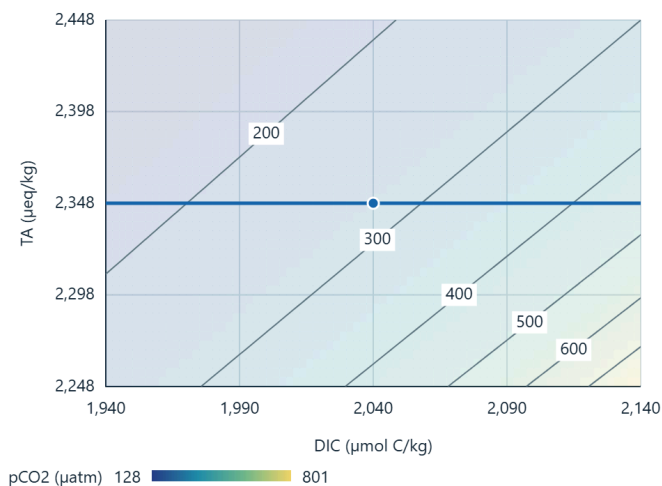
Range

Near reference seawater

TA 2,348.2 $\mu\text{eq/kg}$ DIC point 2,040 $\mu\text{mol C/kg}$ ☐ Show local tangent

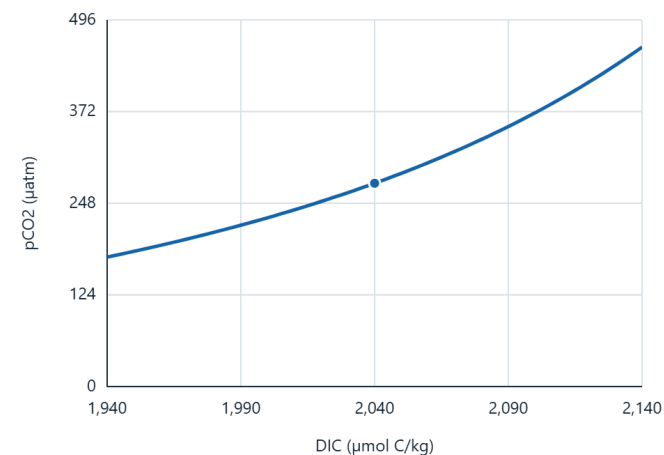
☐ Add conserved-carbon atm line

DIC-TA plane: constant- $p\text{CO}_2$ contours



Contour labels are $p\text{CO}_2$ in μatm . The blue horizontal line is the selected TA slice.

$p\text{CO}_2$ -DIC at the selected TA



The right-hand curve has its own linear $p\text{CO}_2$ scale. Select Full range to include TA = 0; near-reference seawater then looks flatter.

Selected state: DIC 2,040 $\mu\text{mol C/kg}$; TA 2,348.2 $\mu\text{eq/kg}$; seawater $p\text{CO}_2$ 275.08 μatm . Local sensitivity at fixed TA: 1.308 $\mu\text{atm per } (\mu\text{mol C/kg})$.

Each point is a possible carbonate-equilibrium state, not a time step or necessarily air-sea equilibrium. Fixed T/S/P and chemistry choices apply to every view. Height, colour and contours encode $p\text{CO}_2$; pH would require a different surface.

DIC and TA constrain carbonate chemistry

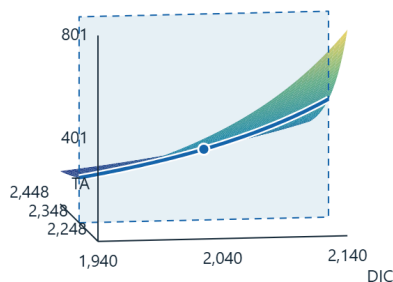
16 °C · salinity 35 · 0 bar · carbonate constants 10 · pH scale 2

1 · Rotate surface
2 · Top view / contours
3 · Constant-TA slice
Reset
Range
Near reference seawater

TA
DIC point
Show local tangent
Add conserved-carbon atm line

Seawater pCO₂ surface + constant-TA plane

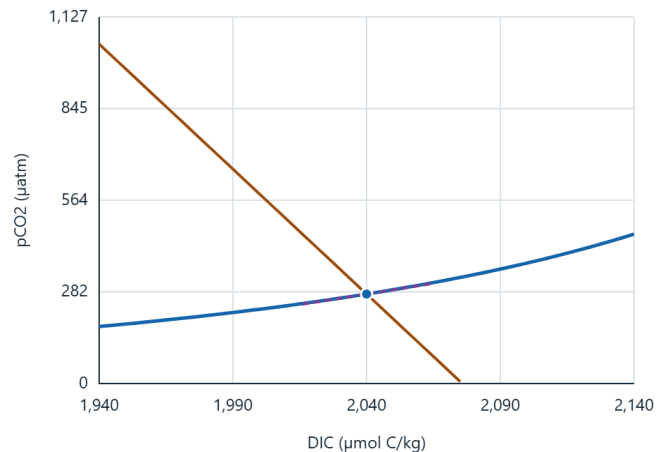
Height: pCO₂ (μatm)



DIC: μmol C/kg · TA: μeq/kg
pCO₂ (μatm) 128 801

Drag or use arrow keys to rotate. The blue curve is the surface cut by the vertical constant-TA plane.

pCO₂-DIC at the selected TA



ocn: selected TA local tangent atm: conserved carbon

The atm line adds the closed carbon inventory; it ends where atmospheric carbon would become negative. Axis scale adjusts to fit both curves.

Selected state: DIC 2,040 μmol C/kg; TA 2,348.2 μeq/kg; seawater pCO₂ 275.08 μatm. Local sensitivity at fixed TA: 1.308 μatm per (μmol C/kg).

Each point is a possible carbonate-equilibrium state, not a time step or necessarily air-sea equilibrium. Fixed T/S/P and chemistry choices apply to every view. Height, colour and contours encode pCO₂; pH would require a different surface.

3.3 Calculate the slices and explain the carbon partition

Now calculate two constant-TA slices yourself using PyCO₂SYN. The supplied loop evaluates TA = 0 and your inferred TA over the same DIC range.

The slope of each seawater curve is its **absolute sensitivity**,

$$\left(\frac{\partial pCO_{2,ocn}}{\partial DIC} \right)_{TA,T,S,P}$$

It measures the pCO₂ increase per small DIC increase at fixed TA and thermodynamic conditions, here in μatm per (μmol/kg). The tangent in the explorer shows this local slope. It can vary along a curve; use the calculated curves to compare the two TA cases.

Supplied atmosphere line. At each DIC, conservation requires

$$C_{atm} = C_0 - m_{ocn}DIC, \quad xCO_{2,atm} = \frac{C_{atm}}{N_{atm}}.$$

Here DIC is in mol/kg, m_{ocn} is the ocean mass in kg and N_{atm} is the atmospheric amount in mol. The supplied [atmospheric pco2 curve](#) applies these equations and converts dry-air xCO₂ to pCO₂ using the shared gas settings. Both seawater and atmosphere curves use pCO₂ in μatm; the reference **280 ppm is dry-air xCO₂**, not 280 μatm. The same line applies to both TA cases because their total carbon and reservoir sizes are identical. You can now turn on **Add conserved-carbon atm line** in the explorer: its intersection with the selected seawater slice is an air-sea equilibrium consistent with this inventory. Chemistry states beyond the available carbon remain possible in general, but the atm

line stops before they would require negative atmospheric carbon. The following full-range plots retain both TA cases for comparison.

Question — calculate the seawater curves (Exercise 01.2)

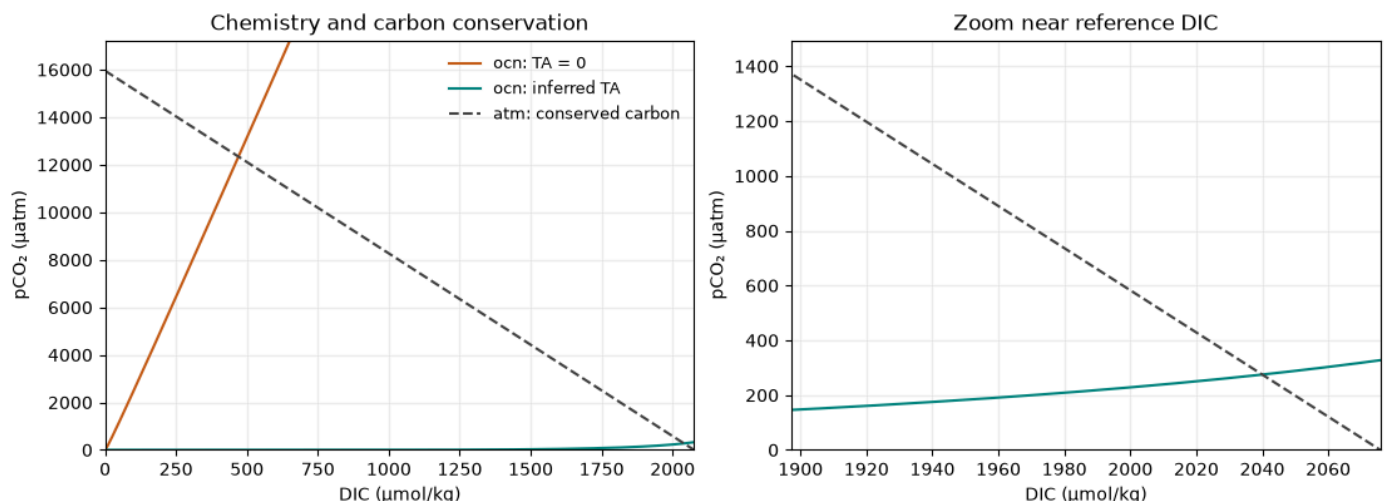
Complete the chemistry calculation inside the supplied loop. For the current `ta_value`, pass `dic_grid` and TA to PyCO2SYS with the shared `**config.pyco2` settings. Choose the input-type codes and save the output `pCO2` array as `curve_pco2`, in `µatm`.

Hints: use the DIC and TA entries in the [input-type table](#) and the `pco2` output. An array can replace a single concentration in a PyCO2SYS call; one TA value applies to every DIC in that array. The supplied loop repeats your call for TA = 0 and `inferred_ta`, then saves both curves in a dictionary. The grid ends just below the limit where all carbon would be in the ocean, leaving no atmospheric carbon.

```
# Supplied implementation: candidate DIC values and the conserved-carbon atmosphere.
max_dic = config.total_carbon_mol / box_mass_kg(M.Ocean) * 1e6
dic_grid = np.linspace(INITIAL_DIC, max_dic * (1 - 1e-6), 600)
atm_pco2 = atmospheric_pco2_curve(dic_grid, config)

# Choose and explain: calculate seawater pCO2 from DIC and TA.
ocean_pco2 = {}
for label, ta_value in [('TA = 0', FIRST_TA), ('inferred TA', inferred_ta)]:
    # BEGIN SOLUTION
    curve = pyco2.sys(par1=dic_grid, par1_type=2,
                     par2=ta_value, par2_type=1, **config.pyco2)
    curve_pco2 = curve['pCO2']
    # END SOLUTION
    ocean_pco2[label] = curve_pco2

# Supplied implementation: linear axes; the right panel enlarges the reference region.
plot_equilibrium_curves(dic_grid, ocean_pco2, atm_pco2, config);
```



Question — explain the contrasting carbon uptake

1. Why does atmospheric $p\text{CO}_2$ fall as DIC increases? Use conservation to relate a given DIC increase to the carbon lost by the atmosphere.
2. Locate each seawater curve's intersection with the atmosphere line and estimate its DIC. Why does each intersection represent zero **net** air–sea exchange? Use the zoom for the inferred-TA case.
3. Relate each curve to a constant-TA slice through the surface and its tangent to local sensitivity. Compare the seawater slopes at the same DIC on the left panel. Starting from the same initial DIC and total carbon, explain the difference in net ocean uptake. Do these curves alone tell you which model equilibrates sooner?

Hint: use the sign of $p\text{CO}_{2,\text{atm}} - p\text{CO}_{2,\text{ocn}}$ to identify the direction of net transfer. For the same ocean mass and initial DIC, a larger final DIC means more net carbon uptake. Compare slopes on the same panel: the zoom has different axis scales. Curves outside a panel's vertical range are clipped.

Instructor answer

Each mole gained by the ocean is lost from the atmosphere. A DIC increase ΔDIC therefore removes $m_{\text{ocn}}\Delta\text{DIC}$ mol from the atmosphere (DIC in mol/kg), lowering atmospheric $x\text{CO}_2$ and $p\text{CO}_2$. This relation is the same in both cases; TA changes the seawater chemistry curve.

The zero-TA intersection is near **470 $\mu\text{mol/kg}$** . The inferred-TA intersection is at **2040 $\mu\text{mol/kg}$** , where $p\text{CO}_2$ is about **275 μatm** , equivalent to the reference dry-air $x\text{CO}_2$ of **280 ppm**. At an intersection the two $p\text{CO}_2$ values are equal, so invasion and outgassing balance and net exchange is zero. The second intersection recovers the state used to fit TA, not an independent prediction of it.

Each seawater curve holds TA fixed while DIC varies: it is the intersection of the chemistry surface with a constant-TA plane. The tangent slope gives the local absolute sensitivity; the top-view contours show the same $p\text{CO}_2$ changes along a horizontal slice.

For these settings and this DIC range, the zero-TA seawater curve rises more steeply (higher absolute sensitivity) at a given DIC. Starting from the common small initial DIC and $p\text{CO}_2$, it meets the falling atmosphere line at much lower DIC, after less net carbon transfer. With inferred TA, seawater $p\text{CO}_2$ stays lower over this range and more carbon can accumulate before air–sea equilibrium is reached. The full curves and the common inventory constraint establish this result; one local derivative alone would not determine the endpoint. The initial DIC and atmospheric carbon are identical, but the initial chemical speciation differs with TA.

This explains the **amount** absorbed. The curves have no time axis and do not establish which case equilibrates sooner; section 4 examines exchange rate and equilibration time.

Connection to the lecture. The Revelle sensitivity factor R expresses the local $p\text{CO}_2$ –DIC response in fractional terms. On [slide 46](#), it estimates fractional DIC uptake for a prescribed atmospheric CO_2 increase at fixed TA and thermodynamic conditions. The doubling example is a linear approximation using fixed R . Here we use the full curves and our closed carbon inventory.

3.4 Test the curve interpretation with a fresh model

Keep the same total carbon, geometry, initial DIC and piston velocity. Change only the initial TA. Compare the final DIC with the intersection you identified above. This is a **calibration and software-consistency check**, not an independent prediction of atmospheric $x\text{CO}_2$.

The supplied [single_box](#) helper repeats section 2's construction: create a model, initialize the ocean, add carbonate chemistry, and connect the atmosphere. It returns a **fresh, unrun model**; `run_model` then runs it. Thus `buffered` is a separate experiment with a different initial TA inventory, not an alkalinity addition during the first run.

Later, the same helper changes initial DIC or piston velocity. Changing initial DIC adjusts atmospheric carbon automatically to preserve the total inventory.

```
# Understand and run: test the revised initial TA with the supplied model.
buffered = single_box(ta_umol_kg=inferred_ta)
run_model(buffered)
# Supplied verification: conservation and recovery of the fitted reference state.
print(audit(buffered))
np.testing.assert_allclose(buffered.CO2_At.c[-1] * 1e6, 280, atol=0.5)
np.testing.assert_allclose(buffered.Ocean.DIC.c[-1] * 1e6, 2040, atol=0.2)
```

ESBMTK 0.14.3.1.post0

Copyright (C) 2020 - 2026 Ulrich G. Wortmann

This program comes with ABSOLUTELY NO WARRANTY

This is free software, and you are welcome to redistribute it
under certain conditions; See the LICENSE file for details.

If you use ESBMTK for your research, please cite:

Wortmann et al. 2025, <https://doi.org/10.5194/gmd-18-1155-2025>

status=0

message=The solver successfully reached the end of the integration interval.

Execution took 0.75 CPU seconds, wall time = 0.78 seconds

{'carbon_relative_error': 1.6030773768583089e-15, 'ta_change_mol': 0.0}

4. Compare paths and endpoints

Question — predict the comparisons

Predict the effect of two separate changes to the buffered model:

- Start with **1000 $\mu\text{mol/kg}$ ocean DIC**, keeping total carbon and TA fixed.
- Halve the **piston velocity**, keeping the initial partition and TA fixed.

Then run the supplied comparison. Compare the curves and printed **equilibration times (1% criterion)**. This is the first saved time after which atmospheric CO₂ remains within 1% of its final simulated value. It depends on the starting state and the chosen tolerance; it is not an exponential relaxation constant. The code checks carbon and TA at every saved time.

Question — interpret the comparisons

Which change alters the initial partition, and which alters the exchange rate? Does either change the eventual equilibrium? Explain why the two changes can alter the path while preserving the endpoint.

Instructor answer

Changing initial DIC redistributes the same carbon between atmosphere and ocean. Changing piston velocity alters the exchange rate. At fixed total carbon, TA, geometry and chemistry, neither change alters the gas-exchange equilibrium; both runs recover the buffered reference state. A smaller positive piston velocity lengthens the adjustment.

```
# Understand and run: trace this supplied step and interpret its evidence.
partition = single_box(ta_umol_kg=inferred_ta, initial_dic_umol_kg=1000)
slower = single_box(ta_umol_kg=inferred_ta, piston_velocity='2 m/d')
for case in (partition, slower):
    run_model(case)
    # Supplied verification: prescribed carbon inventory and shared endpoint.
    carbon, _ = inventories(case)
    np.testing.assert_allclose(carbon, config.total_carbon_mol, rtol=2e-6)
    np.testing.assert_allclose(case.CO2_At.c[-1], buffered.CO2_At.c[-1], atol=0.5e-6)
    # audit also checks time-resolved carbon and TA conservation.
    print(audit(case))

plot_partition_comparison(buffered, partition, slower)
```

ESBMTK 0.14.3.1.post0

Copyright (C) 2020 - 2026 Ulrich G.Wortmann

This program comes with ABSOLUTELY NO WARRANTY

This is free software, and you are welcome to redistribute it
under certain conditions; See the LICENSE file for details.

If you use ESBMTK for your research, please cite:

Wortmann et al. 2025, <https://doi.org/10.5194/gmd-18-1155-2025>

ESBMTK 0.14.3.1.post0

Copyright (C) 2020 - 2026 Ulrich G.Wortmann

This program comes with ABSOLUTELY NO WARRANTY

This is free software, and you are welcome to redistribute it
under certain conditions; See the LICENSE file for details.

If you use ESBMTK for your research, please cite:

Wortmann et al. 2025, <https://doi.org/10.5194/gmd-18-1155-2025>

status=0

message=The solver successfully reached the end of the integration interval.

Execution took 0.64 CPU seconds, wall time = 0.63 seconds

{'carbon_relative_error': 2.137436502477746e-15, 'ta_change_mol': 0.0}

status=0

message=The solver successfully reached the end of the integration interval.

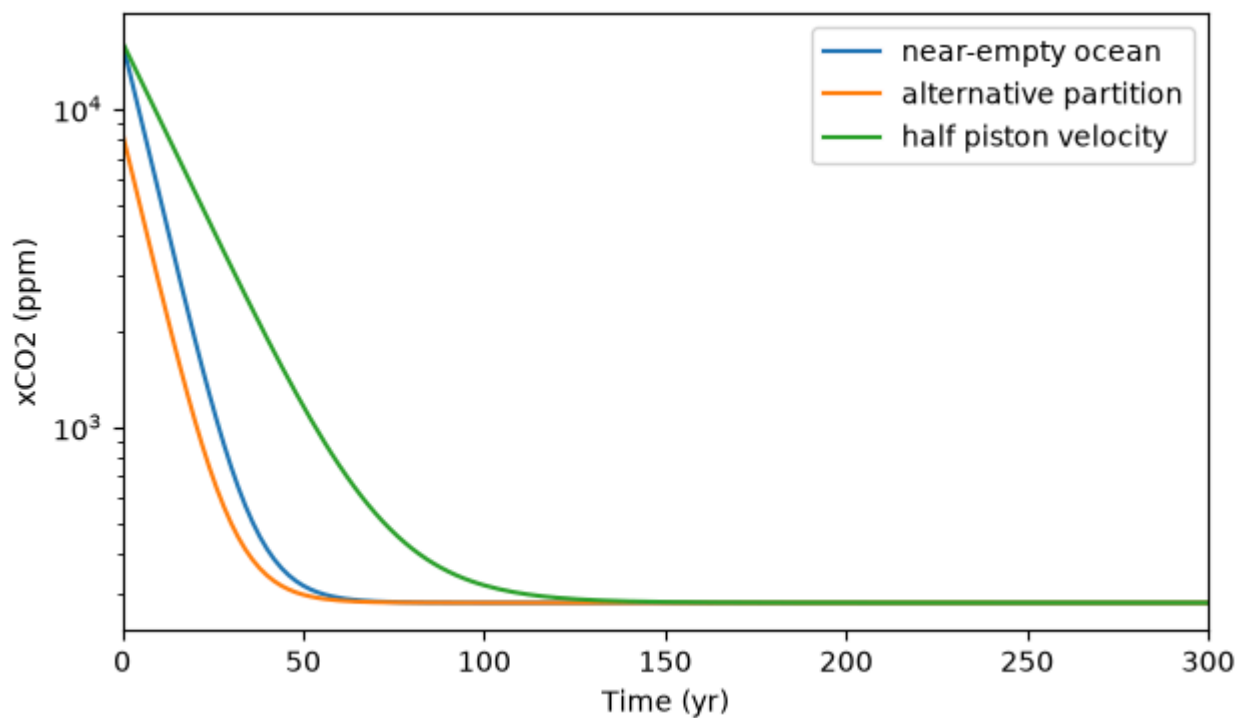
Execution took 0.66 CPU seconds, wall time = 0.66 seconds

{'carbon_relative_error': 1.2468379597786846e-15, 'ta_change_mol': 0.0}

near-empty ocean equilibration time (yr; 1% criterion): 72.0

alternative partition equilibration time (yr; 1% criterion): 64.0

half piston velocity equilibration time (yr; 1% criterion): 142.0



5. Continue to the two-layer model

Real-ocean TA reflects weathering, mineral dissolution and carbonate burial. This closed model includes none of those sources or sinks; prescribing background TA does not simulate its origin. Those processes enter explicitly in 03/04.

You have completed 01. Keep your TA calculation and explanations in this notebook; no separate submission is required. Continue to 02, which retains the same total carbon inventory and adds a deep-ocean reservoir.

Numerical reference. The small positive initial DIC does not supply alkalinity. ESBMTK's carbonate-system-1 approximation can show transient pH discrepancies in extreme states (paper section 2.4). Here we check conserved inventories and stationary agreement; the extreme transient pH is not a realistic seawater history. The helper supplies the native gas-exchange routine's factor-of-1000 conversion; no additional student calculation is required.

```
# Supplied implementation: instructor verification of the expected outcomes.
# Excluded from the student copy so students interpret the results themselves.
from teaching_plots import equilibration_time
assert abs(M.CO2_At.c[-1] * 1e6 - config.target_xco2_ppm) > 1000
assert equilibration_time(slower) > equilibration_time(buffered)
```